

TRƯỜNG ĐẠI HỌC AN GIANG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

Chuyên đề Python

**XÂY DỰNG ỨNG DỤNG QUẢN LÝ ĐIỂM
SINH VIÊN VỚI PYTHON, TKINTER VÀ
MYSQL**

Giảng viên hướng dẫn: Ths. Nguyễn Ngọc Minh

Sinh viên thực hiện:

DTH235677.Trần Duy Khánh.DH24TH2_Nhóm TH2_Tổ TH2

DTH235717.Bùi Thành Nhơn.DH24TH2_Nhóm TH2_Tổ TH2

AN GIANG, 09 - 2025

LỜI NÓI ĐẦU

Trong bối cảnh cuộc Cách mạng công nghiệp 4.0 đang tác động mạnh mẽ đến mọi mặt của đời sống xã hội, chuyển đổi số trong lĩnh vực giáo dục đã trở thành một xu thế tất yếu. Việc ứng dụng công nghệ thông tin vào công tác quản lý đào tạo không chỉ giúp nâng cao hiệu quả hoạt động mà còn góp phần xây dựng một môi trường giáo dục hiện đại và minh bạch.

Tuy nhiên, trên thực tế, công tác quản lý điểm số sinh viên tại nhiều cơ sở đào tạo vẫn còn dựa trên các phương pháp thủ công như sử dụng sổ sách giấy tờ hoặc bảng tính Excel. Các phương pháp truyền thống này bộc lộ nhiều hạn chế nghiêm trọng: tốn nhiều thời gian và công sức của cán bộ giáo vụ, dữ liệu dễ xảy ra sai sót trong quá trình nhập liệu và tính toán, đồng thời việc lưu trữ, tra cứu và thống kê kết quả học tập gặp rất nhiều khó khăn, thiếu tính nhất quán.

Nhằm giải quyết những bất cập nêu trên và góp phần vào quá trình hiện đại hóa công tác quản lý giáo dục, đề tài “Xây dựng ứng dụng quản lý điểm sinh viên bằng Python, Tkinter và MySQL” được thực hiện. Mục tiêu của đề tài là phát triển một công cụ phần mềm chuyên dụng, có giao diện đồ họa thân thiện và cơ sở dữ liệu mạnh mẽ, giúp tự động hóa quy trình quản lý điểm một cách nhanh chóng – chính xác – hiệu quả. Ứng dụng sẽ là một giải pháp hữu ích, hỗ trợ đắc lực cho giảng viên và phòng giáo vụ trong việc theo dõi, đánh giá kết quả học tập của sinh viên một cách khoa học và tiện lợi.

MỤC LỤC

LỜI NÓI ĐẦU	I
MỤC LỤC	Error! Bookmark not defined.
DANH SÁCH HÌNH ẢNH	IV
CHƯƠNG 1. ĐẶT VẤN ĐỀ	1
1.1 Tính cần thiết đề tài	1
1.1.1 Vấn đề thực tế	1
1.1.2 Giải pháp	1
1.1.3 Ý nghĩa lý luận	1
1.1.4 Ý nghĩa thực tiễn	1
1.2 Mục tiêu nghiên cứu	2
1.2.1 Mục tiêu tổng quát	2
1.2.2 Mục tiêu cụ thể	2
1.3 Logic tổng thể của chương trình	2
1.4 Tính khả thi và mở rộng	3
CHƯƠNG 2. TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT	5
2.1 Đặt vấn đề	5
2.1.1 Bối cảnh	5
2.1.2 Nội dung bài toán cần giải quyết	5
2.2 Lịch sử giải quyết vấn đề	5
2.3 Phạm vi của đề tài	6
2.3.1 Phạm vi chức năng (Toàn bộ vấn đề hay một phần)	6
2.3.2 Đối tượng sử dụng	6
2.3.3 Phạm vi triển khai	6
2.4. Phương pháp nghiên cứu	6
2.4.1 Phương pháp nghiên cứu lý thuyết	6
2.4.2 Phương pháp/Hướng giải quyết (Phát triển phần mềm)	7
2.5 Trình bày tóm tắt cơ sở lý thuyết	7
2.5.1 Cơ sở lý thuyết kế thừa	7

2.5.2 Phần mới xây dựng	7
CHƯƠNG 3. KẾT QUẢ ĐẠT ĐƯỢC VÀ DEMO CODE	8
3.1 KẾT QUẢ ĐẠT ĐƯỢC	8
3.1.1 Về giao diện và thiết kế	8
3.1.2 Về chức năng	8
3.1.3 Giao diện đăng nhập	9
3.2 Vận dụng	10
3.2.1 Chức năng và ý nghĩa kỹ thuật	10
3.2.2 Đánh giá hiệu quả	11
A. THIẾT KẾ GIAO DIỆN QUẢN LÝ NHÂN VIÊN	12
A.1 Form đăng nhập	12
A.2 Form quản lý điểm sinh viên	12
B. CÁC BƯỚC XÂY DỰNG ỨNG DỤNG	16
1. Cài đặt các thư viện cần thiết	16
2. Thiết kế cơ sở dữ liệu MySQL	16
3. Xây dựng giao diện người dùng (GUI) bằng Tkinter	19
4. Viết mã Python kết nối cơ sở dữ liệu	20
5. Xây dựng giao diện Tkinter	25
6. Form đăng nhập giảng viên (Login Form)	26
6.1 Mục đích	26
6.2 Giao diện đăng nhập	27
C. HƯỚNG DẪN CÀI ĐẶT VÀ CODE ỨNG DỤNG	28
1. Cài đặt môi trường	28
2. Chuẩn bị Cơ sở dữ liệu (CSDL) MySQL	28
3. Viết code ứng dụng với TKinter + MySQL	32
3.1 Form đăng nhập - giảng viên	32
3.2 Form quản lý điểm sinh viên	34
PHỤ LỤC	43
TÀI LIỆU THAM KHẢO	43

DANH SÁCH HÌNH ẢNH

Hình 1: Giao diện đăng nhập giảng viên.....	11
Hình 2: Giao diện quản lý điểm sinh viên.....	11
Hình 3: Giao diện danh sách lớp.....	12
Hình 4: Giao diện danh sách sinh viên.....	12
Hình 5: Giao diện danh sách khoa.....	13
Hình 6: Giao diện danh sách giảng viên.....	13
Hình 7: Giao diện danh sách môn học.....	14
Hình 8: Bảng khoa.....	15
Hình 9: Bảng Lớp.....	15
Hình 10: Bảng Giảng Viên.....	15
Hình 11: Bảng sinh viên.....	16
Hình 12: Bảng môn học.....	16
Hình 13: Bảng điểm.....	17
Hình 14: Sơ đồ ERD.....	17
Hình 15: Hàm kết nối Mysql.....	19
Hình 16: Hàm thêm điểm.....	19
Hình 17: Hàm sửa điểm.....	19
Hình 18: Hàm xóa điểm.....	20
Hình 19: Hàm lưu điểm.....	20
Hình 20: Hàm tính điểm trung bình và xếp loại.....	21
Hình 21: Hàm tải dữ liệu.....	21
Hình 22: Hàm tìm kiếm điểm sinh viên.....	22

CHƯƠNG 1. ĐẶT VẤN ĐỀ

1.1 Tính cần thiết đề tài

1.1.1 Vấn đề thực tế

Trong môi trường giáo dục hiện nay, việc quản lý điểm sinh viên vẫn thường được thực hiện bằng file Excel hoặc ghi chép thủ công, dẫn đến các vấn đề:

- Sai sót khi nhập liệu: giáo viên nhập nhầm mã sinh viên, sai môn học, sai điểm.
- Khó khăn khi thống kê: khi cần tính điểm trung bình, xếp loại hoặc xuất danh sách học lực phải làm thủ công.
- Không có tính bảo mật: dữ liệu có thể bị mất, sửa đổi hoặc xóa nhầm.
- Không có giao diện người dùng trực quan: người quản lý phải thao tác nhiều bước, khó sử dụng.

1.1.2 Giải pháp

Đề tài hướng đến việc xây dựng một ứng dụng quản lý điểm sinh viên có giao diện đồ họa, xử lý logic bằng Python (Tkinter) và cơ sở dữ liệu MySQL, nhằm:

- Tự động hóa toàn bộ quy trình nhập, sửa, xóa, tính điểm, xếp loại.
- Đảm bảo dữ liệu được lưu trữ tập trung, tránh trùng lặp và mất mát.
- Giúp người dùng thao tác nhanh gọn, chỉ bằng vài nút bấm.

1.1.3 Ý nghĩa lý luận

Ứng dụng mô hình 3 lớp (GUI – Logic – Database):

- Giao diện (Tkinter): hiển thị & nhận thao tác người dùng.
- Xử lý (Python): kiểm tra, tính toán, xử lý nghiệp vụ.
- Dữ liệu (MySQL): lưu trữ, cập nhật và truy xuất thông tin.

Tích hợp các kiến thức về:

- Cấu trúc dữ liệu và thuật toán (xử lý danh sách, sắp xếp, tìm kiếm).
- Lập trình hướng đối tượng.
- Kết nối và thao tác cơ sở dữ liệu (SQL, CRUD).

1.1.4 Ý nghĩa thực tiễn

- Giúp giảng viên quản lý điểm hiệu quả hơn, không cần dùng Excel.

- Có thể mở rộng thành hệ thống quản lý sinh viên hoàn chỉnh (có module quản lý lớp, môn, khoa, giảng viên).
- Giảm thời gian thao tác, tăng độ chính xác, dễ dàng thống kê học lực và kết quả học tập.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Phát triển một phần mềm quản lý điểm sinh viên có khả năng:

- Nhập, sửa, xóa, tìm kiếm và hiển thị điểm.
- Tự động tính điểm trung bình và xếp loại học lực.
- Dễ sử dụng, ổn định, chính xác và có thể mở rộng.

1.2.2 Mục tiêu cụ thể

- Kết nối cơ sở dữ liệu MySQL: Tạo hàm connect_db() để kết nối; toàn bộ lệnh CRUD đều sử dụng qua connection này
- Quản lý thông tin sinh viên, lớp, khoa, môn học, giảng viên: Bảng dữ liệu tương ứng: sinhvien, lop, khoa, monhoc, giangvien
- Quản lý điểm (thêm, sửa, xóa, xem danh sách, tìm kiếm): Giao diện có các nút: “Thêm điểm”, “Sửa điểm”, “Xóa điểm”, “Tải danh sách”, “Tìm kiếm”, “Tính trung bình và xếp loại”, “Reset”, “Thoát”, “Lưu điểm”.
- Tính điểm trung bình và xếp loại: Hàm calc_average() lấy điểm từ bảng diem, tính AVG(diemTong) và phân loại học lực
- Giao diện để thao tác: Tkinter chia layout theo frame, menu điều hướng giữa các trang
- Bảo đảm tính khả thi: Sử dụng thư viện chuẩn Tkinter, MySQL Connector, để chạy trên mọi máy có Python
- Tối ưu trải nghiệm người dùng: Có nút “Reset” để xóa nhanh form, “Thoát” để đóng chương trình, bảng TreeView hiển thị trực quan

1.3 Logic tổng thể của chương trình

Mô hình hoạt động

- Người dùng mở chương trình: Giao diện chính (Tkinter)
- Người dùng đăng nhập vào from đăng nhập: Tên đăng nhập và mật khẩu đã được thiết lập trong cơ sở dữ liệu. Sau đó hiển thị ra from quản lý điểm sinh viên.

- Người dùng chọn chức năng từ Menu: Giao diện “Quản lý điểm sinh viên” (frame "diem") được hiển thị.

- Người dùng nhập thông tin (mã SV, mã môn, điểm QT, điểm Thi): Khi bấm “Thêm điểm”, dữ liệu được kiểm tra rồi lưu vào MySQL qua hàm `add_diem()`.

- Khi bấm “Tải danh sách”: Gọi `load_data("diem")`, dữ liệu được truy xuất từ DB và hiển thị trong TreeView.

- Khi bấm “Tính điểm TB & Xếp loại”: Hàm `calc_average()` truy vấn trung bình `AVG(diemTong)` từ MySQL → tự động phân loại “Giỏi / Khá / Trung bình / Yếu”.

- Khi chọn một dòng trong bảng: Sự kiện `select_item()` tự động điền ngược thông tin vào các ô nhập để người dùng có thể sửa hoặc xóa.

- Khi người dùng chọn một dòng trong bảng (Treeview), sự kiện `select_item()`: Tự động điền dữ liệu dòng đó lên các ô nhập (Entry, Combobox). Sau khi chỉnh sửa, nhấn “Sửa điểm”: Gọi hàm `sua_diem()`, sau đó Cập nhật lại dòng tương ứng trong MySQL bằng lệnh `UPDAT` và làm mới bảng hiển thị (`load_data("diem")`).

- Người dùng chọn dòng cần xóa → nhấn nút “Xóa điểm”. Gọi hàm `xoa_diem()`: Lấy mã SV và mã MH từ dòng được chọn, Hiển thị hộp thoại xác nhận. xóa dòng tương ứng khỏi MySQL (`DELETE FROM diem WHERE maSV=%s AND maMH=%s`) và làm mới bảng hiển thị.

- Người dùng nhập từ khóa (mã SV, tên môn, ...) vào ô tìm kiếm. Nhấn “Tìm kiếm” → gọi hàm `tim_kiem_diem()`: Thực hiện câu lệnh SQL `SELECT ... WHERE ... LIKE %tukhoa%` và hiển thị kết quả lọc trong Treeview.

- Khi bấm “Reset” hoặc “Thoát”: Giao diện được làm mới hoặc chương trình kết thúc.

1.4 Tính khả thi và mở rộng

- Đề tài đủ nhỏ gọn để hoàn thành trong phạm vi môn học, nhưng cũng đủ thực tế để mở rộng thành hệ thống quản lý học vụ.

Có thể nâng cấp thêm các module:

- Quản lý đăng ký môn học.
- Xuất bảng điểm ra Excel/PDF.
- Phân quyền người dùng (sinh viên / giảng viên / quản trị).

- Giao diện hiện đại hơn với customtkinter hoặc PyQt5.

CHƯƠNG 2. TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

2.1 Đặt vấn đề

2.1.1 Bối cảnh

- Các hệ thống quản lý điểm hiện tại (kể cả các phần mềm đang được sử dụng) thường chỉ dừng lại ở vai trò một "kho lưu trữ số". Chúng nhập điểm, tính trung bình và xuất báo cáo. Tuy nhiên, trong kỷ nguyên số, dữ liệu điểm không chỉ là con số, mà là một tài sản quý giá có thể nói lên câu chuyện về năng lực, xu hướng và tiềm năng của sinh viên. Các hệ thống hiện tại đang bỏ lỡ cơ hội khai thác "câu chuyện" này.

2.1.2 Nội dung bài toán cần giải quyết

- Vấn đề chính: Các phương pháp quản lý điểm truyền thống không còn đáp ứng được yêu cầu về hiệu quả, độ chính xác và tính tiện dụng trong môi trường giáo dục hiện nay.

- Hậu quả: Sai sót trong việc nhập liệu, tính toán điểm trung bình; khó khăn trong việc lưu trữ và truy xuất dữ liệu lâu dài; mất nhiều thời gian cho việc làm báo cáo, thống kê; sinh viên không chủ động theo dõi được kết quả học tập của mình.

- Giải pháp đề xuất: Cần xây dựng một hệ thống phần mềm quản lý điểm sinh viên tập trung, cho phép các đối tượng (Giảng viên, Sinh viên, Phòng đào tạo) có thể tương tác với dữ liệu điểm một cách hiệu quả, chính xác và an toàn.

2.2 Lịch sử giải quyết vấn đề

Vấn đề này không mới và đã được giải quyết ở nhiều cấp độ:

- Giai đoạn đầu (thủ công): Các trường đại học sử dụng sổ điểm giấy. Giảng viên ghi điểm vào sổ, sau đó nộp cho phòng đào tạo để tổng hợp.

+ Kết quả: Dễ thất lạc, sai sót khi tổng hợp, khó tra cứu.

- Giai đoạn tin học hóa cơ bản: Sử dụng phần mềm bảng tính như Microsoft Excel để quản lý điểm cho từng lớp học phần.

+ Kết quả: Cải thiện việc tính toán tự động nhưng dữ liệu phân mảnh, không nhất quán, khó tổng hợp toàn trường và không có cơ chế phân quyền, bảo mật.

- Giai đoạn phát triển phần mềm chuyên dụng: Nhiều trường đã tự phát triển hoặc mua các phần mềm quản lý đào tạo trong đó có chức năng quản lý điểm.

+ Kết quả: Đây là giải pháp phổ biến nhất hiện nay. Tuy nhiên, nhiều hệ thống cũ có giao diện không thân thiện, thiếu các tính năng phân tích nâng cao,

không tương thích với các thiết bị di động, hoặc chi phí triển khai và bảo trì cao.

Điểm còn tồn tại hoặc hướng phát triển mới của đề tài:

- Xây dựng hệ thống trên nền tảng web/cloud hiện đại, dễ dàng truy cập mọi lúc mọi nơi.
- Tích hợp module phân tích dữ liệu, cảnh báo học vụ sớm cho - những sinh viên có kết quả học tập yếu.
- Phát triển ứng dụng di động (mobile app) cho sinh viên và giảng viên để tăng tính tiện lợi.
- Tối ưu hóa giao diện người dùng (UI/UX) để mang lại trải nghiệm tốt nhất.

2.3 Phạm vi của đề tài

2.3.1 Phạm vi chức năng (Toàn bộ vấn đề hay một phần)

- Đề tài sẽ tập trung xây dựng các chức năng cốt lõi của việc quản lý điểm, không bao gồm các mảng khác như quản lý tuyển sinh, quản lý học phí hay quản lý thư viện.

2.3.2 Đối tượng sử dụng

- Sinh viên: Xem điểm chi tiết các môn học, xem điểm trung bình tích lũy (GPA), theo dõi tiến độ học tập.
- Giảng viên: Nhập/xuất bảng điểm của lớp học phần mình phụ trách, chỉnh sửa điểm (trong thời gian cho phép).
- Quản trị viên (Phòng đào tạo): Quản lý thông tin sinh viên, môn học, lớp học; tổng hợp điểm, xét tốt nghiệp; in bảng điểm và các báo cáo thống kê.

2.3.3 Phạm vi triển khai

- Hệ thống được xây dựng thử nghiệm cho một khoa cụ thể (ví dụ: Khoa Công nghệ thông tin) hoặc áp dụng cho toàn bộ sinh viên hệ đại học chính quy của một trường giả định.

2.4. Phương pháp nghiên cứu

2.4.1 Phương pháp nghiên cứu lý thuyết

- Thu thập và phân tích yêu cầu: Khảo sát, phỏng vấn các đối tượng người dùng (sinh viên, giảng viên, cán bộ đào tạo) để hiểu rõ nhu cầu và những khó khăn họ đang gặp phải.
- Nghiên cứu các hệ thống tương tự: Phân tích ưu, nhược điểm của các phần mềm quản lý điểm hiện có trên thị trường để học hỏi và rút kinh nghiệm.

2.4.2 Phương pháp/Hướng giải quyết (Phát triển phần mềm)

- Mô hình phát triển: Áp dụng mô hình phát triển phần mềm linh hoạt như Agile/Scrum để dễ dàng điều chỉnh theo yêu cầu.
- Công nghệ sử dụng:
 - Phía Server (Back-end): Sử dụng ngôn ngữ lập trình Java (Spring Boot) hoặc PHP (Laravel) để xử lý logic nghiệp vụ.
 - Cơ sở dữ liệu (Database): Sử dụng hệ quản trị CSDL0020 quan hệ như MySQL hoặc PostgreSQL để lưu trữ dữ liệu.
 - Phía Client (Front-end): Sử dụng framework ReactJS hoặc VueJS để xây dựng giao diện người dùng tương tác, thân thiện.
- Kiến trúc hệ thống: Xây dựng theo kiến trúc Model-View-Controller (MVC) để tách biệt các thành phần của ứng dụng, dễ dàng phát triển và bảo trì.

2.5 Trình bày tóm tắt cơ sở lý thuyết

2.5.1 Cơ sở lý thuyết kế thừa

- Hệ quản trị cơ sở dữ liệu: Vận dụng kiến thức về mô hình dữ liệu quan hệ, ngôn ngữ truy vấn SQL, thiết kế và tối ưu hóa CSDL (ví dụ: các dạng chuẩn hóa để tránh dư thừa dữ
- Quy chế đào tạo theo hệ thống tín chỉ: Nghiên cứu và áp dụng đúng các công thức tính điểm trung bình môn học, điểm trung bình tích lũy (GPA), các quy định về số tín chỉ tối thiểu/tối đa, điều kiện xét tốt nghiệp... của Bộ Giáo dục & Đào tạo hoặc của một trường đại học cụ thể. Đây là "logic nghiệp vụ" cốt lõi của hệ thống.
- Các nguyên lý trong Công nghệ phần mềm: Áp dụng các kiến thức về quy trình phát triển phần mềm, các mô hình kiến trúc (MVC), các mẫu thiết kế (design patterns).

2.5.2 Phần mới xây dựng

- Dựa trên các cơ sở lý thuyết trên, ta sẽ tiến hành thiết kế mô hình cơ sở dữ liệu riêng cho bài toán quản lý điểm (bao gồm các bảng như SinhVien, MonHoc, LopHocPhan, Diem...).
- Xây dựng các thuật toán để xử lý nghiệp vụ đặc thù như: tính GPA theo thang điểm 4, xếp loại học lực, cảnh báo học vụ tự động dựa trên kết quả học tập.
- Thiết kế giao diện người dùng mới, đảm bảo tính khoa học và dễ sử dụng cho từng đối tượng người dùng.

CHƯƠNG 3. KẾT QUẢ ĐẠT ĐƯỢC VÀ DEMO CODE

3.1 KẾT QUẢ ĐẠT ĐƯỢC

Sau thời gian nghiên cứu và lập trình, sinh viên đã hoàn thành đề tài “Quản lý điểm sinh viên” bằng ngôn ngữ Python, sử dụng thư viện Tkinter để xây dựng giao diện và MySQL để quản lý dữ liệu.

Ứng dụng hoạt động ổn định, có đầy đủ các chức năng cơ bản và thể hiện được tư duy logic, khả năng thiết kế phần mềm quản lý.

3.1.1 Về giao diện và thiết kế

Giao diện chính

- Ứng dụng có menu chọn chức năng: Quản lý sinh viên, Lớp, Khoa, Môn học, Giảng viên, và đặc biệt là Quản lý điểm.
- Mỗi phần là một Frame riêng biệt, dễ bảo trì và mở rộng.
- Màu sắc nhẹ nhàng, bố cục cân đối, giúp người dùng dễ thao tác.

Trang “Quản lý điểm”

- Hiện thị các ô nhập liệu: Mã sinh viên, mã môn học, điểm quá trình, điểm thi
- Có các nút chức năng: Thêm điểm, sửa điểm, xóa điểm, tính điểm trung bình và xếp loại, tải danh sách, tìm kiếm, reset / thoát

Bảng hiển thị dữ liệu (Treeview)

- Dùng ttk.Treeview để hiển thị danh sách điểm sinh viên.
- Có các cột: Mã SV, Mã Môn, Điểm QT, Điểm Thi, Điểm TB, Xếp loại.
- Bảng tự động cập nhật sau khi thêm, sửa hoặc xóa điểm.
- Kết hợp cuộn dọc (Scrollbar) để xem danh sách dài.

3.1.2 Về chức năng

Thêm điểm

Khi người dùng nhập đủ thông tin và nhấn “Thêm điểm”, chương trình sẽ:

- Kiểm tra dữ liệu hợp lệ (điểm phải là số, trong khoảng 0–10).
- Gửi câu lệnh:

```
INSERT INTO diem (maSV, maMH, diemQT, diemThi) VALUES (%s, %s, %s, %s)", (maSV, maMH, diemQT, diemThi)
```

- Cập nhật ngay bảng Treeview.

Ý nghĩa: Thể hiện kỹ năng kết nối cơ sở dữ liệu và xử lý sự kiện trong Tkinter.

Sửa điểm

Khi người dùng chọn một dòng trong bảng, dữ liệu được hiển thị lại lên các ô nhập.

Sau khi chỉnh sửa, nhấn “Sửa điểm”, ứng dụng:

- Gửi câu lệnh:

```
UPDATE diem SET diemQT=%s, diemThi=%s WHERE maSV=%s AND maMH=%s", (diemQT, diemThi, maSV, maMH))
```

- Hiển thị thông báo thành công.

Ý nghĩa: Giúp người dùng chỉnh sửa điểm sai dễ dàng, đảm bảo tính linh hoạt của phần mềm.

Xóa điểm

- Cho phép chọn dòng trong Treeview để xóa.
- Hiển thị thông báo xác nhận (messagebox.askyesno).
- Thực hiện câu lệnh:

```
DELETE FROM diem WHERE maSV=%s AND maMH=%s", (maSV, maMH)
```

Ý nghĩa: Đảm bảo khả năng làm sạch dữ liệu và tránh sai sót nhờ xác nhận người dùng.

3.1.3 Giao diện đăng nhập

Giao diện “Đăng nhập giảng viên” được thiết kế bằng Tkinter kết hợp PIL (Pillow) để chèn ảnh nền, giúp phần mở đầu của phần mềm có giao diện trực quan và chuyên nghiệp hơn.

Mục tiêu của form này là xác thực người dùng giảng viên trước khi truy cập hệ thống chính.

Cấu trúc giao diện

- Cửa sổ chính: được tạo bằng tk.Tk() với kích thước cố định 700x500, căn giữa màn hình.
- Ảnh nền: chèn bằng ImageTk.PhotoImage() giúp giao diện sinh động hơn.
- Khung đăng nhập (Frame): có màu nền nhẹ (#FBFBF2), bố trí các ô nhập liệu và nút lệnh gọn gàng giữa màn hình.

- Các thành phần chính:
 - + Entry nhập tên đăng nhập và mật khẩu (ẩn ký tự bằng show="*").
 - + Button “Đăng nhập” gọi hàm check_login() khi nhấn

Trải nghiệm người dùng:

- Giao diện thân thiện, thao tác đơn giản: chỉ cần nhập tài khoản và mật khẩu.
- Có thông báo trực quan (messagebox) khi đăng nhập thành công hoặc thất bại.
- Nếu đăng nhập thành công, form tự đóng và mở giao diện chính của chương trình (DoAn_Python_QL_DiemSinhVien).
- Nếu sai, hiển thị thông báo lỗi, giúp người dùng dễ dàng nhận biết và sửa thông tin.

3.2 Vận dụng

- Ứng dụng đã vận dụng hiệu quả các kỹ thuật lập trình Python như xử lý sự kiện, thao tác cơ sở dữ liệu MySQL và thiết kế giao diện bằng Tkinter.
- Giải quyết được bài toán thực tế trong công tác quản lý điểm sinh viên: hỗ trợ thêm, sửa, xóa, tìm kiếm và thống kê điểm nhanh chóng, chính xác.
- Góp phần giảm tải công việc thủ công, nâng cao hiệu quả và tính chính xác trong quản lý học tập của nhà trường.

3.2.1 Chức năng và ý nghĩa kỹ thuật

Hàm check_login() thể hiện sự kết hợp giữa giao diện và cơ sở dữ liệu MySQL.

Quy trình xử lý:

- Lấy dữ liệu nhập từ người dùng bằng entry_user.get() và entry_pass.get().
- Kiểm tra dữ liệu rỗng → nếu thiếu, hiển thị cảnh báo bằng messagebox.showwarning.
- Kết nối cơ sở dữ liệu MySQL thông qua hàm connect_db().
- Thực hiện truy vấn kiểm tra thông tin đăng nhập:

```
# Thực hiện truy vấn an toàn (dùng tham số hóa để tránh SQL injection)
cur.execute("SELECT * FROM giangvien WHERE hoTen=%s AND matkhau=%s", (username, password))
```

→ đảm bảo bảo mật nhờ truy vấn tham số hóa (tránh SQL Injection).

- Phản hồi kết quả:

- + Nếu tồn tại tài khoản: hiển thị “Đăng nhập thành công”.
- + Nếu sai: hiển thị “Sai tên đăng nhập hoặc mật khẩu”.

Ý nghĩa:

Thể hiện khả năng kết nối, truy vấn và xử lý dữ liệu MySQL trong Python.

Biết tích hợp giao diện (Tkinter) với logic xử lý dữ liệu thực tế.

Là bước khởi đầu bắt buộc để bảo mật hệ thống, chỉ cho phép người có quyền truy cập.

3.2.2 Đánh giá hiệu quả

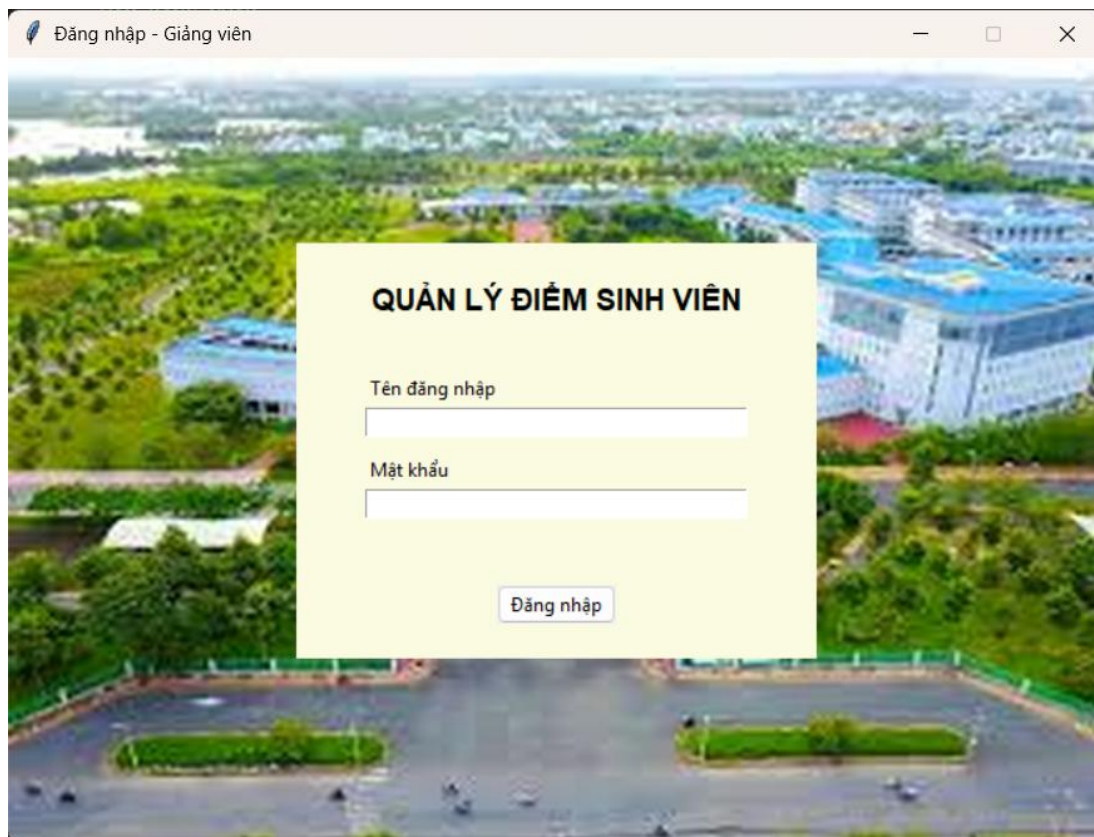
Giao diện đơn giản, gọn nhẹ, dễ hiểu cho người dùng phổ thông (giảng viên).

Dễ dàng mở rộng để phân quyền thêm (ví dụ: sinh viên, quản trị viên).

Giúp ứng dụng mang tính chuyên nghiệp và hoàn chỉnh, không chỉ là công cụ nhập/xuất dữ liệu.

A. THIẾT KẾ GIAO DIỆN QUẢN LÝ NHÂN VIÊN

A.1 Form đăng nhập



Đăng nhập - Giảng viên

QUẢN LÝ ĐIỂM SINH VIÊN

Tên đăng nhập

Mật khẩu

Đăng nhập

Hình 1: Giao diện đăng nhập giảng viên

A.2 Form quản lý điểm sinh viên



Quản lý điểm sinh viên

Trang chính

QUẢN LÝ ĐIỂM SINH VIÊN

Mã số sinh viên: Mã môn học:

Điểm quá trình: Điểm thi:

Thêm điểm Sửa điểm Xóa điểm Tính điểm TB & Xếp loại Tải danh sách

Lưu điểm Reset Tìm kiếm Thoát

Mã Số Sinh Viên	Mã Môn Học	Điểm Quá Trình	Điểm Thi	Điểm Tổng
DAV235696	MH03	7.0	8.7	8.02
DAV237847	MH05	7.0	6.5	6.7
DBT233295	MH02	6.0	9.5	8.1
DBT233295	MH08	8.0	9.5	8.9
DGT237096	MH05	9.0	9.5	9.3
DKT239858	MH04	6.0	8.0	7.2
DMK236120	MH10	8.0	9.0	8.6
DMK237859	MH03	7.0	9.0	8.2
DMK237859	MH13	7.0	6.0	6.4
DPM236537	MH07	8.5	7.0	7.6
DTU235677	MH01	8.0	8.0	8.0

Hình 2: Giao diện quản lý điểm sinh viên

Quản lý điểm sinh viên		
Trang chính		
DANH SÁCH LỚP		
Mã Lớp	Tên Lớp	Mã Khoa
DH24AV1	Sử phạm tiếng anh - 01	SP
DH24AV2	Sử phạm tiếng anh - 02	SP
DH24BT1	Bảo vệ thực vật - 02	NN
DH24BT2	Bảo vệ thực vật - 02	NN
DH24GT	Giáo dục tiểu học	SP
DH24KT	Kế toán	KT
DH24MK1	Marketing - 01	KT
DH24MK2	Marketing - 02	KT
DH24PM1	Kỹ thuật phần mềm	CNTT
DH24PM2	Kỹ thuật phần mềm	CNTT
DH24TH2	Công nghệ thông tin - 02	CNTT
DH24TY	Thú y	NN

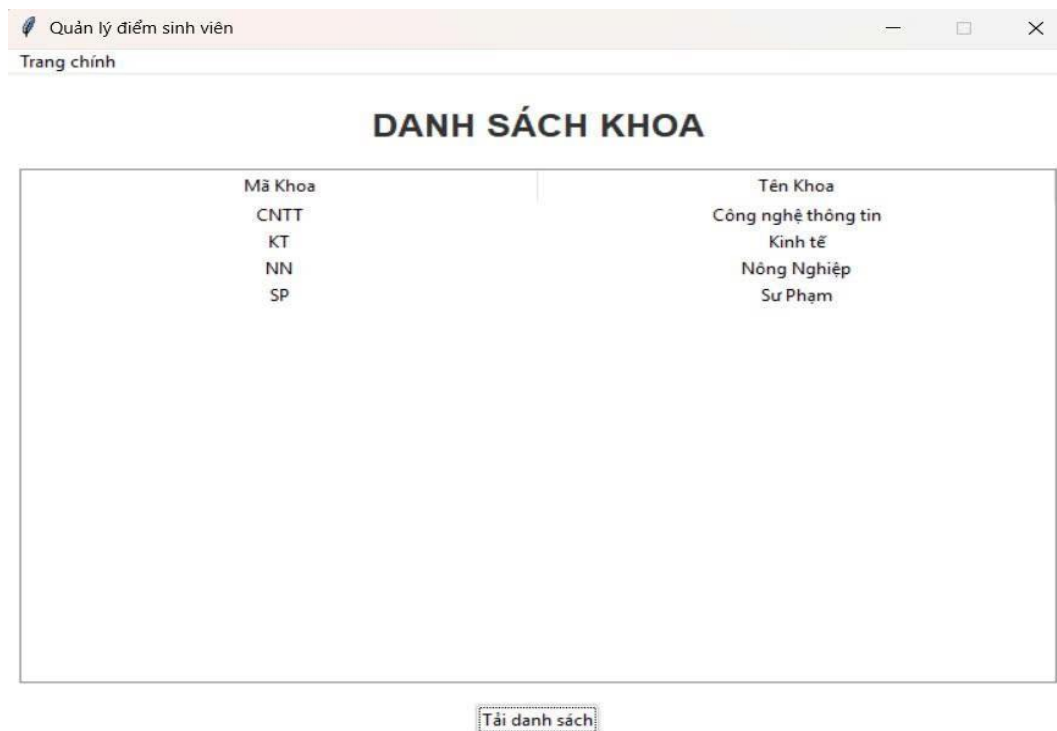
Tải danh sách

Hình 3: Giao diện danh sách lớp

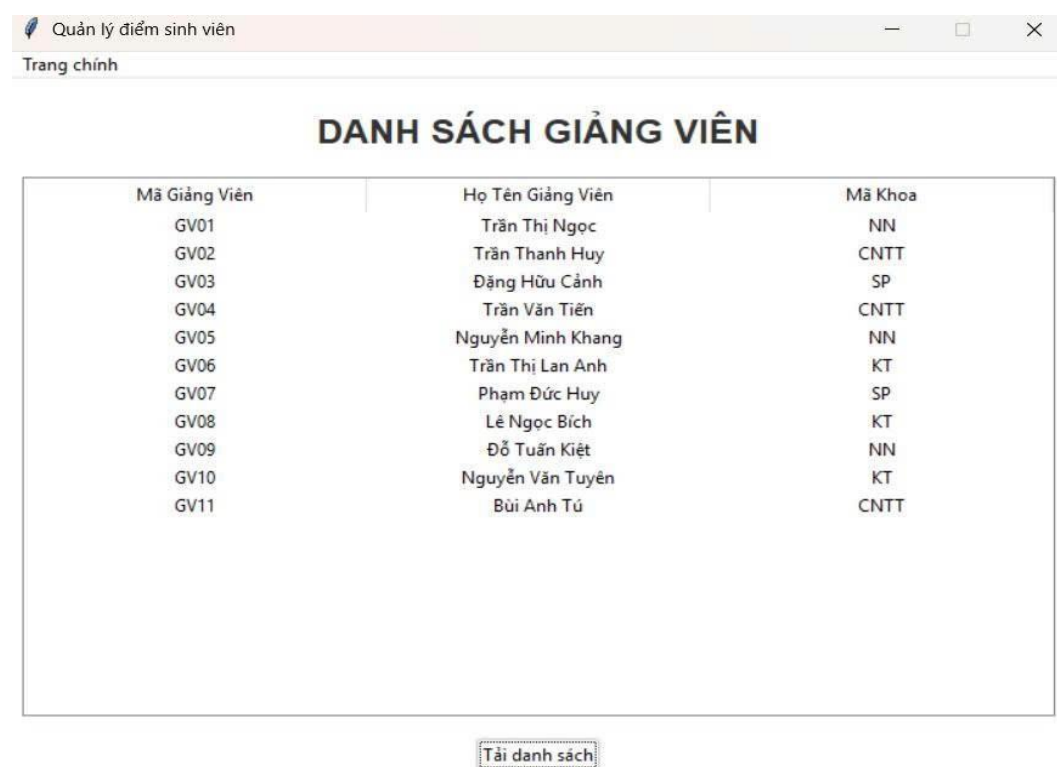
Quản lý điểm sinh viên						
Trang chính						
DANH SÁCH SINH VIÊN						
Mã Số Sinh Viên	Họ Tên	Ngày Sinh	Giới Tính	Địa Chỉ	Mã Lớp	
DAV235696	Nguyễn Hữu Chương	2005-01-01	Nam	An Giang	DH24AV2	
DAV237847	Nguyễn Trí Tài	2005-01-28	Nam	An Giang	DH24AV1	
DBT233295	Đỗ Thanh Vy	2005-06-08	Nữ	Đồng Tháp	DH24BT1	
DGT237096	Nguyễn Hoàng Nan	2005-01-12	Nam	Lạng Sơn	DH24GT	
DKT239858	Nguyễn Bảo Minh	2005-12-25	Nam	Kiên Giang	DH24KT	
DMK236120	Nguyễn Minh Truờ	2005-09-08	Nam	Vĩnh Long	DH24MK2	
DMK237859	Nguyễn Trọng Nghi	2005-11-06	Nam	Hà Nội	DH24MK1	
DPM236537	Trần Hoài Phương	2005-05-16	Nam	TP.Hồ Chí Minh	DH24PM1	
DTH235677	Trần Duy Khánh	2005-09-10	Nam	An Giang	DH24TH2	
DTH237612	Bùi Thành Nhơn	2005-06-06	Nam	An Giang	DH24PM2	
DTY239270	Diệp Yến Vy	2005-01-28	Nữ	TP.Hồ Chí Minh	DH24TY	

Tải danh sách

Hình 4: Giao diện danh sách sinh viên



Hình 5: Giao diện danh sách khoa



Hình 6: Giao diện danh sách giảng viên

Quản lý điểm sinh viên			
Trang chính			
DANH SÁCH MÔN HỌC			
Mã Môn Học	Tên Môn Học	Số Tín Chỉ	Giảng Viên Phụ Trách
MH01	Cơ sở dữ liệu	3	GV02
MH02	Tuyển trùng	4	GV01
MH03	Toán c	2	GV06
MH04	Kinh tế chính trị	3	GV08
MH05	Kỹ năng giảng dạy	4	GV03
MH06	Kiến trúc máy tính	3	GV11
MH07	Lý thuyết đồ thị	3	GV04
MH08	Cây ăn quả	4	GV09
MH09	Ngữ pháp	2	GV07
MH10	Xác xuất thống kê	3	GV08
MH11	Lập trình cân bằng	4	GV11
MH12	Sinh học đại cương	4	GV05
MH13	Tài chính ngân hàng	3	GV10
Tải danh sách			

Hình 7: Giao diện danh sách môn học

B. CÁC BƯỚC XÂY DỰNG ỨNG DỤNG

Để xây dựng ứng dụng quản lý điểm sinh viên bằng Python, Tkinter và MySQL, cần cài đặt các thư viện cần thiết, thiết kế cơ sở dữ liệu MySQL để lưu trữ thông tin sinh viên, môn học, giảng viên và điểm số, sử dụng Tkinter để tạo giao diện người dùng, và kết hợp Python với thư viện mysql.connector để thực hiện các thao tác như thêm, xóa, sửa, và hiển thị bảng điểm.

1. Cài đặt các thư viện cần thiết

- Python: Đảm bảo bạn đã cài đặt Python trên máy tính của mình.
- MySQL: Cài đặt máy chủ MySQL (ví dụ: MySQL Community Server) và tạo một cơ sở dữ liệu (schema) cho ứng dụng của bạn (ví dụ: ql_diemsinhvien).
- Mysql-connector-python: Cài đặt thư viện này để kết nối Python với cơ sở dữ liệu MySQL.

```
inhVien.py'  
PS D:\DTH235677_PyThon\DoAn_Python.NhomDoAn12.DH24TH2_NhomTH02_ToTH02> pip install mysql-connector-python tkcalendar
```

2. Thiết kế cơ sở dữ liệu MySQL

- Bảng khoa: Lưu trữ thông tin về các khoa (ví dụ: CNTT, Kinh tế).

```
-- bảng khoa  
create table khoa(  
    maKhoa char(10) primary key,  
    tenKhoa varchar(100) not null  
);
```

Hình 8: Bảng khoa

- Bảng lop: Lưu trữ thông tin các lớp học, mỗi lớp thuộc một khoa.

```
-- bảng lớp  
create table lop(  
    maLop char(10) primary key,  
    tenLop varchar(100) not null,  
    maKhoa char(10),  
    foreign key (maKhoa) references khoa(maKhoa)  
);
```

Hình 9: Bảng Lớp

- Bảng giangvien: Lưu trữ thông tin giảng viên, mỗi giảng viên thuộc một khoa.

```
-- bảng giảng viên
CREATE TABLE giangvien (
    maGV VARCHAR(10) PRIMARY KEY,
    hoTen VARCHAR(100) NOT NULL,
    maKhoa VARCHAR(10),
    matKau VARCHAR(50) NOT NULL,
    FOREIGN KEY (maKhoa) REFERENCES khoa(maKhoa)
);
```

Hình 10: Bảng Giảng Viên

- Bảng sinhvien: Lưu trữ thông tin chi tiết của sinh viên, mỗi sinh viên thuộc một lớp.

```
-- bảng sinh viên
create table sinhvien (
    maSV char(10) primary key,
    hoTen varchar(100) not null,
    ngaySinh date,
    gioiTinh enum ( 'Nam', 'Nữ'),
    diaChi varchar(200),
    maLop char(10),
    foreign key (maLop) references lop(maLop)
);
```

Hình 11: Bảng sinh viên

- Bảng monhoc: Lưu trữ thông tin các môn học, mỗi môn học do một giảng viên phụ trách.

```
-- bảng môn học
CREATE TABLE monhoc (
    maMH CHAR(10) PRIMARY KEY,
    tenMH VARCHAR(100) NOT NULL,
    soTinChi INT NOT NULL,
    maGV CHAR(10),
    FOREIGN KEY (maGV) REFERENCES giangvien(maGV)
);
```

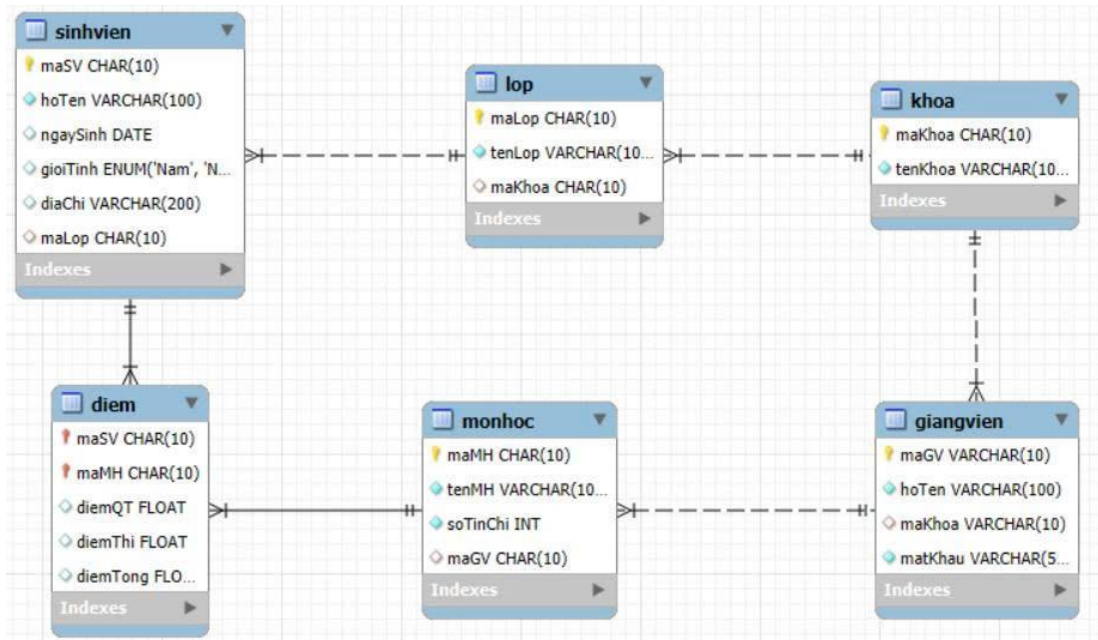
Hình 12: Bảng môn học

- Bảng diem: Bảng trung tâm lưu trữ điểm thi, điểm quá trình của sinh viên cho từng môn học. Bảng này liên kết trực tiếp sinhvien và monhoc.

```
-- bảng điểm
CREATE TABLE diem (
    maSV CHAR(10),
    maMH CHAR(10),
    diemQT FLOAT CHECK (diemQT BETWEEN 0 AND 10),
    diemThi FLOAT CHECK (diemThi BETWEEN 0 AND 10),
    diemTong FLOAT GENERATED ALWAYS AS ((diemQT * 0.4) + (diemThi * 0.6)) STORED,
    PRIMARY KEY (maSV, maMH),
    FOREIGN KEY (maSV) REFERENCES sinhvien(maSV),
    FOREIGN KEY (maMH) REFERENCES monhoc(maMH)
);
```

Hình 13: Bảng điểm

- Sơ đồ ERM:



Hình 14: Sơ đồ ERD

3. Xây dựng giao diện người dùng (GUI) bằng Tkinter

Sử dụng thư viện Tkinter để xây dựng giao diện chương trình Quản lý điểm sinh viên, cho phép người dùng thao tác trực quan với dữ liệu trong cơ sở dữ liệu.

Các thành phần của giao diện:

- Cửa sổ chính (root window): Có tiêu đề “Quản lý điểm sinh viên”, kích thước cố định, được căn giữa màn hình.
- Menu chuyển trang (cho phép người dùng di chuyển giữa các trang khác nhau):
 - + Quản lý điểm sinh viên
 - + Danh sách sinh viên
 - + Danh sách lớp
 - + Danh sách khoa
 - + Danh sách giảng viên
 - + Danh sách môn học
- Các trường nhập dữ liệu:

- + Mã sinh viên
- + Mã môn học
- + Điểm quá trình
- + Điểm thi
- Các nút chức năng:
 - + Thêm điểm sinh viên mới
 - + Sửa điểm sinh viên
 - + Xóa điểm sinh viên
 - + Reset (hủy nhập dữ liệu hiện tại)
 - + Lưu danh sách ra Excel
 - + Tính điểm trung bình và xếp loại
 - + Tải danh sách từ CSDL
 - + Tìm kiếm sinh viên
 - + Thoát chương trình
- Bảng hiển thị dữ liệu (Treeview): Hiển thị danh sách sinh viên, môn học và điểm được lấy từ cơ sở dữ liệu ql_diemsinhvien. Khi người dùng nhấn chọn 1 hàng, dữ liệu sẽ được điền vào các ô nhập để chỉnh sửa hoặc xóa.

4. Viết mã Python kết nối cơ sở dữ liệu

Sử dụng thư viện mysql.connector để kết nối với CSDL MySQL ql_diemsinhvien, thực hiện các chức năng quản lý điểm sinh viên.

Các hàm chính:

- Hàm kết nối Mysql (connect_db()):

```
# ===== HÀM KẾT NỐI MYSQL =====
def connect_db():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="Khanh@091025",
        database="ql_diemsinhvien"
        #hàm này để mở kết nối đến database ql_diemsinhvien.
    )
```

Hình 15: Hàm kết nối Mysql

- Hàm thêm điểm (them_diem()):
 - + Lấy dữ liệu từ các trường nhập (maSV, maMH, diemQT, diemThi)
 - + Tính điểm tổng = $0.4 * \text{diemQT} + 0.6 * \text{diemThi}$
 - + Ghi vào bảng diem trong cơ sở dữ liệu.

```
# ==== HÀM THÊM ====
def them_diem():
    try:
        maSV = maSV_entry.get()
        maMH = maMH_entry.get()
        diemQT = float(diemQT_entry.get()) #Lấy dữ liệu từ Entry, ép kiểu float cho điểm.
        diemThi = float(diemThi_entry.get())
        diemTong = diemQT * 0.4 + diemThi * 0.6

        conn = connect_db()
        cur = conn.cursor()
        cur.execute("INSERT INTO diem (maSV, maMH, diemQT, diemThi) VALUES (%s, %s, %s, %s)", (maSV, maMH, diemQT, diemThi))
        conn.commit()
        conn.close()
        messagebox.showinfo("Thành công", "Đã thêm điểm!")
        load_data("diem", tree_diem) #Gọi load_data("diem", tree_diem) để refresh bảng hiển thị sau khi thao tác.
    except Exception as e:
        messagebox.showerror("Lỗi", str(e))
```

Hình 16: Hàm thêm điểm

- Hàm sửa điểm (sua_diem()):
 - + Cập nhật điểm quá trình và điểm thi của sinh viên trong bảng diem theo maSV và maMH.

```
# ==== HÀM SỬA ====
def sua_diem(): #dùng UPDATE ... WHERE maSV=%s AND maMH=%s để cập nhật điểm.
    try:
        maSV = maSV_entry.get()
        maMH = maMH_entry.get()
        diemQT = float(diemQT_entry.get())
        diemThi = float(diemThi_entry.get())
        diemTong = diemQT * 0.4 + diemThi * 0.6
        conn = connect_db()
        cur = conn.cursor()
        cur.execute("UPDATE diem SET diemQT=%s, diemThi=%s WHERE maSV=%s AND maMH=%s", (diemQT, diemThi, maSV, maMH))
        conn.commit()
        conn.close()
        messagebox.showinfo("Thành công", "Đã cập nhật điểm!")
        load_data("diem", tree_diem) #gọi load_data sau khi commit để cập nhật UI.
    except Exception as e:
        messagebox.showerror("Lỗi", str(e))
```

Hình 17: Hàm sửa điểm

- Hàm xóa điểm (xoa_diem()):
 - + Xóa một dòng điểm khỏi bảng diem theo maSV và maMH.

```
# ==== HÀM XÓA ====
def xoa_diem(): #dùng DELETE FROM diem WHERE maSV=%s AND maMH=%s để xóa.
    try:
        maSV = maSV_entry.get()
        maMH = maMH_entry.get()
        conn = connect_db()
        cur = conn.cursor()
        cur.execute("DELETE FROM diem WHERE maSV=%s AND maMH=%s", (maSV, maMH))
        conn.commit()
        conn.close()
        messagebox.showinfo("Thành công", "Đã xóa điểm!")
        load_data("diem", tree_diem) #gọi load_data sau khi commit để cập nhật UI.
    except Exception as e:
        messagebox.showerror("Lỗi", str(e))
```

Hình 18: Hàm xóa điểm

- Hàm lưu điểm vào file excel (luu_excel()):
 - + Xuất toàn bộ danh sách điểm sinh viên (bao gồm tên SV, môn học, điểm) ra file .xlsx.
 - + Cho phép người dùng chọn nơi lưu bằng hộp thoại `filedialog.asksaveasfilename()`.

```

# ==== HÀM LƯU RA FILE EXCEL ====
def luu_excel():
    try:
        # Hộp thoại chọn nơi lưu
        # asksaveasfilename() -> mở hộp thoại "Save As" cho người dùng
        # defaultextension=".xlsx": nếu không gõ đuôi, tự thêm .xlsx
        # filetype: lọc chỉ hiển thị file Excel
        # title: tiêu đề của số
        file_path = filedialog.asksaveasfilename(
            defaultextension=".xlsx",
            filetype=[("Excel files", "*.xlsx")],
            title="Chọn nơi lưu file Excel"
        )
        if not file_path: #kết quả trả về file_path nếu người dùng bấm cancel thì return để thoát hàm mà không làm gì cả
            return # nếu người dùng bấm Cancel

        # Kết nối DB & lấy dữ liệu
        conn = connect_db()
        cur = conn.cursor()
        cur.execute("""
            SELECT d.maSV, sv.hoTen, d.maMH, mh.tenMH, d.diemQT, d.diemThi, d.diemTong
            FROM diem d
            JOIN sinhvien sv ON d.maSV = sv.maSV
            JOIN monhoc mh ON d.maMH = mh.maMH
            """)
        rows = cur.fetchall()
        conn.close()

        # Ghi vào file Excel
        workbook = xlwt.Workbook(file_path) #Tạo một workbook Excel mới và sẽ ghi vào đường dẫn file_path.
        worksheet = workbook.add_worksheet("Điểm sinh viên") #thêm một sheet (worksheet) mới vào workbook với tên "Điểm sinh viên".
        bold = workbook.add_format({'bold': True, 'bg_color': '#D9E1F2'}) #Tạo định dạng: chữ in đậm và màu nền

        # Tiêu đề cột
        headers = ["Mã SV", "Họ tên", "Mã môn học", "Tên môn học", "Điểm QT", "Điểm Thi", "Điểm Tổng"]
        for col, header in enumerate(headers): #Lập qua từng header cùng chỉ số cột col (bắt đầu từ 0).
            worksheet.write(0, col, header, bold) #Ghi tiêu đề vào hàng 0 (dòng đầu tiên trong Excel), cột col, với format bold (in đậm + màu nền).
            worksheet.set_column(col, col, 18) #Đặt độ rộng của cột

        # Ghi dữ liệu
        for row_num, row_data in enumerate(rows, start=1):
            for col_num, value in enumerate(row_data):
                worksheet.write(row_num, col_num, value)

        workbook.close() #Đóng workbook và ghi thực sự file .xlsx ra đĩa
        messagebox.showinfo("Thành công", f"Đã lưu dữ liệu ra file:\n{file_path}")

    except Exception as e:
        messagebox.showerror("Lỗi", str(e))

```

Hình 19: Hàm lưu điểm

- Hàm tính điểm trung bình và xếp loại (calc_average(tree)):
 - + Tính điểm trung bình và xếp loại học lực của từng sinh viên.
 - + Dữ liệu được nhóm theo maSV và hiển thị trong bảng Treeview.

```
# ===== HÀM TÍNH ĐIỂM TB & XẾP LOẠI =====
def calc_average(tree):
    for item in tree.get_children():
        tree.delete(item)
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT sv.maSV, sv.hoTen, AVG(d.diemTong) AS diemTB,
        CASE
            WHEN AVG(d.diemTong) >= 8 THEN 'Giỏi'
            WHEN AVG(d.diemTong) >= 6.5 THEN 'Khá'
            WHEN AVG(d.diemTong) >= 5 THEN 'Trung bình'
            ELSE 'Yếu'
        END AS xepLoai
        FROM diem d
        JOIN sinhvien sv ON d.maSV = sv.maSV
        GROUP BY sv.maSV, sv.hoTen
    """)
    rows = cursor.fetchall()
    columns = ["Mã SV", "Họ Tên", "Điểm TB", "Xếp loại"]
    tree["columns"] = columns
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=150, anchor="center")
    for row in rows:
        diem_tb = round(row[2], 2) if row[2] is not None else 0 # làm tròn 2 chữ số
        tree.insert("", "end", values=(row[0], row[1], diem_tb, row[3]))
    conn.close()
```

H

ình 20: Hàm tính điểm trung bình và xếp loại

- Hàm tải dữ liệu (load_data(table_name, tree)):
 - + Truy vấn cơ sở dữ liệu để hiển thị danh sách dữ liệu của bảng tương ứng: diem, sinhvien, lop, khoa, giangvien, monhoc.
 - + Hiển thị kết quả trên bảng Treeview giao diện.

```
# ===== HÀM TẢI DỮ LIỆU =====
def load_data(table_name, tree):
    for item in tree.get_children():
        tree.delete(item)
    conn = connect_db() #Mở kết nối database
    cursor = conn.cursor() #tạo cursor để chạy truy vấn.

    query_map = { #là dict ánh xạ tên logical (vd "sinhvien") sang câu SQL tương ứng. Giúp hàm chung xử lý nhiều bảng.
        "khoa": "SELECT maKhoa AS 'Mã Khoa', tenKhoa AS 'Tên Khoa' FROM khoa",
        "lop": "SELECT maLop AS 'Mã Lớp', tenLop AS 'Tên Lớp', maKhoa AS 'Mã Khoa' FROM lop",
        "giangvien": "SELECT maGV AS 'Mã Giảng Viên', hoTen AS 'Họ Tên Giảng Viên', maKhoa AS 'Mã Khoa' FROM giangvien",
        "sinhvien": "SELECT maSV AS 'Mã Số Sinh Viên', hoTen AS 'Họ Tên', ngaySinh AS 'Ngày Sinh', gioiTinh AS 'Giới Tính', diaChi AS 'Địa Chỉ', maLop AS 'Mã Lớp' FROM sinhvien",
        "monhoc": "SELECT maMH AS 'Mã Môn Học', tenMH AS 'Tên Môn Học', soTinChi AS 'Số Tin Chí', maGV AS 'Giảng Viên Phụ Trách' FROM monhoc",
        "diem": "SELECT maSV AS 'Mã Số Sinh Viên', maMH AS 'Mã Môn Học', diemQT AS 'Điểm Quá Trình', diemThi AS 'Điểm Thi', diemTong AS 'Điểm Tổng' FROM diem"
    }

    cursor.execute(query_map[table_name])
    rows = cursor.fetchall()
    #Thực hiện truy vấn và lấy toàn bộ dữ liệu về.

    columns = [desc[0] for desc in cursor.description] #ấy tên cột (để gán header cho Treeview).
    tree["columns"] = columns #đặt cấu hình cột cho Treeview.
    for col in columns:
        tree.heading(col, text=col) #đặt tiêu đề hiển thị.
        tree.column(col, width=150, anchor="center") #đặt chiều rộng và căn giữa nội dung cột.

    for row in rows:
        tree.insert("", "end", values=row) #thêm từng hàng vào Treeview.
    conn.close() #đóng kết nối (rất quan trọng để giải phóng tài nguyên).
```

Hình 21: Hàm tải dữ liệu

- Hàm tìm kiếm điểm sinh viên(tim_kiem_diem()):

```

# --- HÀM TÌM KIẾM THEO MÃ SINH VIÊN ---
def tim_kiem_diem():
    try:
        maSV = maSV_entry.get().strip()
        if not maSV:
            messagebox.showwarning("Cảnh báo", "Vui lòng nhập mã sinh viên cần tìm!")
            return

        conn = connect_db()
        cur = conn.cursor()
        cur.execute("""
            SELECT d.maSV, mh.maMH, d.diemQT, d.diemThi, d.diemTong
            FROM diem d
            JOIN monhoc mh ON d.maMH = mh.maMH
            WHERE d.maSV = %s
            """, (maSV,))
        rows = cur.fetchall()
        conn.close()

        # Xóa dữ liệu cũ trong TreeView
        for item in tree_diem.get_children():
            tree_diem.delete(item)

        # Cập nhật lại cột
        columns = ["Mã SV", "Tên Môn Học", "Điểm QT", "Điểm Thi", "Điểm Tổng"]
        tree_diem["columns"] = columns
        for col in columns:
            tree_diem.heading(col, text=col)
            tree_diem.column(col, width=150, anchor="center")

        # Thêm dữ liệu tìm thấy
        for row in rows:
            tree_diem.insert("", "end", values=row)

        if not rows:
            messagebox.showinfo("Kết quả", f"Không tìm thấy điểm cho sinh viên có mã: {maSV}")

    except Exception as e:
        messagebox.showerror("Lỗi", str(e))

```

Hình 22: Hàm tìm kiếm điểm sinh viên

5. Xây dựng giao diện Tkinter

Liên kết các nút bấm (Button) và sự kiện (ví dụ: chọn một mục trong danh sách) trên giao diện Tkinter với các hàm Python tương ứng để xử lý dữ liệu và tương tác với cơ sở dữ liệu `ql_diemsinhvien`.

Các nút button chức năng:

- Thêm điểm: Khi người dùng nhập đầy đủ thông tin vào các ô (mã số sinh viên, mã môn học, điểm quá trình, điểm thi) và nhấn nút "Thêm điểm", mã Python sẽ lấy các giá trị này và thực thi một câu lệnh INSERT để thêm một bản ghi mới vào bảng điểm sinh viên trong MySQL.
- Tải danh sách: Khi người dùng mở cửa sổ ứng dụng hoặc chọn tab "Quản lý Điểm Sinh viên", mã Python sẽ tự động truy vấn cơ sở dữ liệu (ví dụ: SELECT maSV AS 'Mã Số Sinh Viên', maMH AS 'Mã Môn Học', diemQT AS 'Điểm Quá Trình', diemThi AS 'Điểm Thi',

diemTong AS 'Điểm Tổng' FROM diem), lấy tất cả thông tin điểm Sinh viên và hiển thị chúng dưới dạng một danh sách hoặc bảng (ví dụ: sử dụng ttk.Treeview) trên giao diện Tkinter.

- Tính điểm TB & Xếp loại: Khi người dùng nhập hoặc sửa điểm, chương trình tự động lấy giá trị diemQt và diemThi bằng công thức $diemTong = round(diemQT * 0.4 + diemThi * 0.6, 2)$. Dựa trên điểm tổng chương trình phân loại như sau: ≥ 8.5 là giỏi, 7.0 đến 8.4 là khá, 5.0 đến 6.9 là trung bình, < 5.0 là yếu. Sau đó hiển thị xếp loại và cập nhật vào bảng điểm
- Sửa điểm: Khi người dùng chọn dòng tương ứng trong Treeview thông tin tự động hiển thị vào các Entry người dùng nhập lại giá trị mới cho các ô gọi hàm sua_diem() trong python hàm sẽ thực thi câu lệnh *UPDATE diem SET diemQT=%s, diemThi=%s WHERE maSV=%s AND maMH=%s*, (diemQT, diemThi, maSV, maMH) sau đó tải lại danh sách và hiển thị kết quả
- Xóa điểm: Người dùng chọn dòng trong bảng Treeview. Thực hiện lệnh xóa *DELETE FROM diem WHERE maSV=%s AND maMH=%s*, (maSV, maMH) sau đó tải lại giao diện và hiển thị thông báo đã xóa thành công
- Tìm kiếm: Người dùng nhập từ khóa (mã SV, tên môn, ...) vào ô tìm kiếm. Nhấn “Tìm kiếm” sẽ gọi hàm tim_kiem_diem():. Thực hiện câu lệnh SQL *SELECT ... WHERE ... LIKE %tukhoa%*. Sau đó hiển thị kết quả lọc trong Treeview
- Lưu điểm: Khi người dùng nhấn nút lưu điểm hàm luu_excel() sẽ được gọi và chọn vị trí lưu file dùng filedialog.asksaveasfilename() để chọn nơi lưu, sau đó kết nối bảng điểm với truy vấn dữ liệu, dùng thư viện *xlsxwriter* để tạo file Excel, ghi dữ liệu ra file và thông báo thành công.
- Reset dữ liệu: Khi người dùng nhấn nút reset thì nội dung các ô nhập sẽ được bị xóa và làm sạch giao diện.
- Thoát: Khi nhấn nút thoát, giao diện sẽ đóng cửa sổ chính và ngắt kết nối dữ liệu bảng điểm (nếu còn mở).

6. Form đăng nhập giảng viên (Login Form)

6.1 Mục đích

Form đăng nhập được xây dựng để xác thực người dùng giảng viên trước khi truy cập vào hệ thống “Quản lý điểm sinh viên”. Điều này giúp đảm bảo tính

bảo mật, tránh trường hợp người không có quyền truy cập có thể xem hoặc chỉnh sửa dữ liệu trong cơ sở dữ liệu.

6.2 Giao diện đăng nhập

Form được thiết kế bằng Tkinter kết hợp với Pillow (PIL) để chèn hình nền.

Các đặc điểm chính:

- Cửa sổ chính (root window):
 - + Tiêu đề: “Đăng nhập - Giảng viên”.
 - + Kích thước: 700x500 pixel, cố định, được căn giữa màn hình bằng công thức tính tọa độ x, y dựa trên kích thước màn hình.
- Ảnh nền:
 - + Dùng PIL.Image.open() và ImageTk.PhotoImage() để hiển thị hình ảnh nền (anh_truong.jpg) giúp giao diện sinh động và chuyên nghiệp hơn.
- Khung đăng nhập (Frame):
 - + Màu nền nhẹ #FBFBF2 tạo cảm giác dễ nhìn.
 - + Bên trong khung gồm các ô nhập liệu (Entry) và nút chức năng (Button).
- Các trường nhập liệu (Entry):
 - + Tên đăng nhập (Username) → nhập tên giảng viên.
 - + Mật khẩu (Password) → ẩn ký tự bằng show="*".
- Nút “Đăng nhập” (Button):
 - + Khi nhấn sẽ gọi hàm check_login() để kiểm tra thông tin đăng nhập.

C. HƯỚNG DẪN CÀI ĐẶT VÀ CODE ỨNG DỤNG

Ứng dụng sẽ có:

- Giao diện đăng nhập giảng viên (Tên đăng nhập và mật khẩu).
- Giao diện nhập điểm sinh viên (Mã số sinh viên, Mã môn học, Điểm quá trình, Điểm thi).
- Bảng điểm sinh viên (trang chính), menu chuyên trang gồm các danh sách: Sinh viên, lớp, môn học, khoa, giảng viên.
- Chức năng CRUD (Thêm điểm, Sửa điểm, Xóa điểm, Lưu điểm, Tải danh sách, Tính điểm trung bình và xếp loại, Reset và thoát).
- Dữ liệu lưu trực tiếp vào MySQL Database và file excel(không mất khi tắt ứng dụng).
- Giao diện Tkinter thiết kế chuẩn như hình minh họa ở trên (hình 1).

1. Cài đặt môi trường

Trước tiên, bạn cần cài đặt các thư viện Python cần thiết:

```
inhVien.py'  
PS D:\DTH235677_PyThon\DoAn_Python.NhomDoAn12.DH24TH2_NhomTH02_ToTH02> pip install mysql-connector-python tkcalendar
```

Giải thích:

- mysql-connector-python: Dùng để kết nối ứng dụng Python với cơ sở dữ liệu MySQL.
- tkcalendar: Dùng để tạo tiện ích lịch, giúp người dùng chọn ngàySinh (ngày sinh) cho sinh viên một cách trực quan khi nhập liệu.

2. Chuẩn bị Cơ sở dữ liệu (CSDL) MySQL

Mở MySQL (WorkBench, phpMyAdmin hoặc terminal) rồi tạo database và bảng:

```
-- tạo database  
CREATE DATABASE QuanLy_DiemSinhVien CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
-- sử dụng database vừa tạo  
use QuanLy_DiemSinhVien;
```

```

-- bảng khoa
create table khoa(
    maKhoa char(10) primary key,
    tenKhoa varchar(100) not null
);

INSERT INTO khoa VALUES ('CNTT', 'Công nghệ thông tin'),
                        ('KT', 'Kinh tế'),
                        ('NN', 'Nông Nghiệp'),
                        ('SP', 'Sư Phạm');

-- bảng lớp
create table lop(
    maLop char(10) primary key,
    tenLop varchar(100) not null,
    maKhoa char(10),
    foreign key (maKhoa) references khoa(maKhoa)
);

INSERT INTO lop VALUES ('DH24TH2', 'Công nghệ thông tin - 02', 'CNTT'),
                        ('DH24PM1', 'Kỹ thuật phần mềm - 01', 'CNTT'),
                        ('DH24PM2', 'Kỹ thuật phần mềm - 02', 'CNTT'),
                        ('DH24MK1', 'Marketing - 01', 'KT'),
                        ('DH24MK2', 'Marketing - 02', 'KT'),
                        ('DH24BT1', 'Bảo vệ thực vật - 01', 'NN'),
                        ('DH24BT2', 'Bảo vệ thực vật - 02', 'NN'),
                        ('DH24AV1', 'Sử phạm tiếng anh - 01', 'SP'),
                        ('DH24AV2', 'Sử phạm tiếng anh - 02', 'SP'),
                        ('DH24TY', 'Thú y ', 'NN'),
                        ('DH24KT', 'Kế toán ', 'KT'),
                        ('DH24GT', 'Giáo dục tiểu học', 'SP');

```

```

-- bảng sinh viên
create table sinhvien (
    maSV char(10) primary key,
    hoTen varchar(100) not null,
    ngaySinh date,
    gioiTinh enum ( 'Nam','Nữ'),
    diaChi varchar(200),
    maLop char(10),
    foreign key (maLop) references lop(maLop)
);

INSERT INTO sinhvien VALUES ('DTH235677', 'Trần Duy Khánh', '2005-09-10', 'Nam', 'An Giang', 'DH24TH2'),
    ('DTH237612', 'Bùi Thành Nhơn', '2005-06-06', 'Nam', 'An Giang', 'DH24PM2'),
    ('DPM236537', 'Trần Hoài Phương', '2005-05-16', 'Nam', 'TP.Hồ Chí Minh', 'DH24PM1'),
    ('DMK237859', 'Nguyễn Trọng Nghĩa', '2005-11-6', 'Nam', 'Hà Nội', 'DH24MK1'),
    ('DMK236120', 'Nguyễn Minh Trường', '2005-09-08', 'Nam', 'Vĩnh Long', 'DH24MK2'),
    ('DAV235696', 'Nguyễn Hữu Chương', '2005-01-01', 'Nam', 'An Giang', 'DH24AV2'),
    ('DAV237847', 'Nguyễn Trí Tài', '2005-01-28', 'Nam', 'An Giang', 'DH24AV1'),
    ('DKT239858', 'Nguyễn Bảo Minh', '2005-12-25', 'Nam', 'Kiên Giang', 'DH24KT'),
    ('DBT233295', 'Đỗ Thanh Vy', '2005-06-08', 'Nữ', 'Đồng Tháp', 'DH24BT1'),
    ('DTV239270', 'Diệp Yến Vy', '2005-01-28', 'Nữ', 'TP.Hồ Chí Minh', 'DH24TY'),
    ('DGT237096', 'Nguyễn Hoàng Nam', '2005-01-12', 'Nam', 'Lạng Sơn', 'DH24GT');

-- bảng giảng viên
CREATE TABLE giangvien (
    maGV VARCHAR(10) PRIMARY KEY,
    hoTen VARCHAR(100) NOT NULL,
    maKhoa VARCHAR(10),
    matKau VARCHAR(50) NOT NULL,
    FOREIGN KEY (maKhoa) REFERENCES khoa(maKhoa)
);

INSERT INTO giangvien VALUES ('GV001', 'Trần Duy Khánh', 'CNTT', '091025'),
    ('GV002', 'Đỗ Thị Vy', 'NN', '080603'),
    ('GV003', 'Bùi Thành Nhơn', 'KT', '123456');

```

```

-- bảng môn học
CREATE TABLE monhoc (
    maMH CHAR(10) PRIMARY KEY,
    tenMH VARCHAR(100) NOT NULL,
    soTinChi INT NOT NULL,
    maGV CHAR(10),
    FOREIGN KEY (maGV) REFERENCES giangvien(maGV)
);

INSERT INTO monhoc VALUES ('MH01', 'Cơ sở dữ liệu', 3, 'GV001'),
    ('MH02', 'Tuyển trùg', 4, 'GV002'),
    ('MH03', 'Toán c', 2, 'GV002'),
    ('MH04', 'Kinh tế chính trị', 3, 'GV003'),
    ('MH05', 'Kỹ năng giảng dạy', 4, 'GV003'),
    ('MH06', 'Kiến trúc máy tính', 3, 'GV001'),
    ('MH07', 'Lý thuyết đồ thị', 3, 'GV001'),
    ('MH08', 'Cây ăn quả', 4, 'GV002'),
    ('MH09', 'Ngữ pháp', 2, 'GV003'),
    ('MH10', 'Xác xuất thông kê', 3, 'GV001'),
    ('MH11', 'Lập trình cân bằng ', 4, 'GV001'),
    ('MH12', 'Sinh học đại cương ', 4, 'GV002'),
    ('MH13', 'Tài chính ngân hàng ', 3, 'GV003');

-- bảng điểm
CREATE TABLE diem (
    maSV CHAR(10),
    maMH CHAR(10),
    diemQT FLOAT CHECK (diemQT BETWEEN 0 AND 10),
    diemThi FLOAT CHECK (diemThi BETWEEN 0 AND 10),
    diemTong FLOAT GENERATED ALWAYS AS ((diemQT * 0.4) + (diemThi * 0.6)) STORED,
    PRIMARY KEY (maSV, maMH),
    FOREIGN KEY (maSV) REFERENCES sinhvien(maSV),
    FOREIGN KEY (maMH) REFERENCES monhoc(maMH)
);

INSERT INTO diem (maSV, maMH, diemQT, diemThi) VALUES
    ('DTH235677', 'MH01', 8.0, 9.0),
    ('DTH235677', 'MH06', 8.0, 7.0),
    ('DPM236537', 'MH07', 8.5, 7.0),
    ('DMK237859', 'MH13', 7.0, 6.0),
    ('DMK237859', 'MH03', 7.0, 9.0),
    ('D0T233295', 'MH02', 6.0, 9.5),
    ('D0T233295', 'MH08', 8.0, 9.5),
    ('DTH237612', 'MH11', 5.5, 9.0),
    ('DKT239858', 'MH04', 6.0, 8.0),
    ('DMK236120', 'MH10', 8.0, 9.0),
    ('DTY239270', 'MH12', 5.0, 8.0),
    ('DGT237096', 'MH05', 9.0, 9.5),
    ('DAV235696', 'MH09', 7.0, 8.7),
    ('DAV237847', 'MH05', 7.0, 6.5);

```

3. Viết code ứng dụng với TKinter + MySQL

3.1 Form đăng nhập - giảng viên

```
import tkinter as tk
from tkinter import ttk, messagebox
from PIL import Image, ImageTk
import mysql.connector
```

```
# ===== HÀM KẾT NỐI DATABASE =====
def connect_db():
    # Trả về một đối tượng kết nối tới
    return mysql.connector.connect(
        host="localhost",          # địa
        user="root",              # tên
        password="Khanh@091025",  # mật
        database="quanly_diemsinhvien"
    )
```

```
# ===== HÀM ĐĂNG NHẬP =====
def check_login():
    # Lấy nội dung từ các ô nhập (Entry) và loại bỏ khoảng trắng thừa
    username = entry_user.get().strip()
    password = entry_pass.get().strip()

    # Kiểm tra xem người dùng đã nhập đủ thông tin chưa
    if not username or not password:
        messagebox.showwarning("Cảnh báo", "Vui lòng nhập đầy đủ thông tin!")
        return

    try:
        # Mở kết nối tới database
        conn = connect_db()
        cur = conn.cursor()
        # Thực hiện truy vấn an toàn (dùng tham số hóa để tránh SQL injection)
        cur.execute("SELECT * FROM giangvien WHERE hoTen=%s AND matKau=%s", (username, password))
        result = cur.fetchone() # Lấy một dòng kết quả
        conn.close() # Đóng kết nối sau khi xong

        # Nếu tìm thấy bản ghi khớp thì đăng nhập thành công
        if result:
            messagebox.showinfo("Thành công", f"Xin chào {username}!")
            root.destroy() # Đóng cửa sổ login hoàn toàn
            import DoAn_Python_QL_DiemSinhVien # import module chính (chạy chương trình chính)
        else:
            # Nếu không có bản ghi nào khớp
            messagebox.showerror("Thất bại", "Sai tên đăng nhập hoặc mật khẩu!")

    except Exception as e:
        # Bắt và hiển thị lỗi nếu có vấn đề (ví dụ: không kết nối được DB)
        messagebox.showerror("Lỗi", str(e))
```

```
# ===== GIAO DIỆN LOGIN =====
root = tk.Tk() # Tạo cửa sổ chính của ứng dụng
root.title("Đăng nhập - Giảng viên")
root.geometry("700x500") # Kích thước cửa sổ
root.resizable(False, False) # Không cho phép thay đổi kích thước cửa sổ
```

```
# ===== CĂN GIỮA MÀN HÌNH =====
window_width = 700
window_height = 500
screen_width = root.winfo_screenwidth() # Lấy kích thước màn hình
screen_height = root.winfo_screenheight()
# Tính tọa độ để căn giữa cửa sổ
x = (screen_width // 2) - (window_width // 2)
y = (screen_height // 2) - (window_height // 2)
# Thiết lập geometry với tọa độ x, y để cửa sổ xuất hiện ở vị trí mong muốn
root.geometry(f"{window_width}x{window_height}+{x}+{y}")
```

```
# ===== CHÈN ẢNH NỀN =====
# Mở ảnh từ file
bg_image = Image.open("anh_truong.jpg")
# Thay đổi kích thước ảnh vừa với cửa sổ (resampling để giữ chất lượng)
bg_image = bg_image.resize((window_width, window_height), Image.Resampling.LANCZOS)
# Chuyển sang đối tượng PhotoImage của Tkinter để hiển thị
bg_photo = ImageTk.PhotoImage(bg_image)

# Tạo label chứa ảnh nền và đặt đầy khung cửa sổ
bg_label = tk.Label(root, image=bg_photo)
bg_label.place(x=0, y=0, relwidth=1, relheight=1)

# ===== KHUNG LOGIN =====
# Tạo một khung (Label dùng làm background) để chứa form đăng nhập
login_frame = tk.Label(root, bg="#FBFBF2", width=450, height=300)
# Đặt khung login ở giữa cửa sổ (relx/rely = 0.5 là trung tâm)
login_frame.place(relx=0.5, rely=0.5, anchor="center")
# Label tiêu đề nằm trong khung login
tk.Label(login_frame, text="QUẢN LÝ ĐIỂM SINH VIÊN", font=("Arial", 14, "bold"),
        bg="#FBFBF2", fg="black").pack(pady=20)

# Frame chứa form (tên đăng nhập + mật khẩu)
form_frame = tk.Label(login_frame, bg="#FBFBF2")
form_frame.pack(pady=10, padx=40, fill="x")
```

```

# ===== Tên đăng nhập =====
# Nhãn cho ô nhập tên đăng nhập
tk.Label(form_frame, text="Tên đăng nhập", bg="■" "#FBFBF2", fg="black", anchor="w").pack(fill="x", pady=(0, 2))
# Ô nhập tên đăng nhập (Entry)
entry_user = tk.Entry(form_frame, width=40)
entry_user.pack(fill="x", pady=(0, 10))

# ===== Mật khẩu =====
# Nhãn cho ô nhập mật khẩu
tk.Label(form_frame, text="Mật khẩu", bg="■" "#FBFBF2", fg="black", anchor="w").pack(fill="x", pady=(0, 2))
# Ô nhập mật khẩu, show="*" để che ký tự khi gõ
entry_pass = tk.Entry(form_frame, width=40, show="*")
entry_pass.pack(fill="x", pady=(0, 10))

# Nút đăng nhập (khi bấm sẽ gọi hàm check_login)
btn_login = ttk.Button(login_frame, text="Đăng nhập", command=check_login)
btn_login.pack(pady=20)

# Bắt đầu vòng lặp chính của Tkinter (hiển thị cửa sổ và chờ sự kiện)
root.mainloop()

```

3.2 Form quản lý điểm sinh viên

```

import tkinter as tk
from tkinter import ttk, messagebox, filedialog # ttk
import mysql.connector # thư viện kết nối MySQL
import xlswriter # dùng để xuất dữ liệu ra file Excel

```

```

# ===== HÀM KẾT NỐI MYSQL =====
def connect_db():
    # Trả về một đối tượng kết nối tới MySQL
    return mysql.connector.connect(
        host="localhost",          # địa chỉ máy chủ
        user="root",               # tên người dùng
        password="Khanh@091025",  # mật khẩu
        database="quanly_diemsinhvien"
    )

```

```
# ===== TẠO CỬA SỐ CHÍNH =====
root = tk.Tk() # Tạo cửa sổ chính Tkinter
root.title("Quản lý điểm sinh viên")
root.geometry("700x500") # Kích thước cửa sổ
root.config(bg="#f7f7f7") # Màu nền cửa sổ
root.resizable(False, False) # Không cho phép thay đổi kích thước cửa sổ
```

```
# ===== CĂN GIỮA MÀN HÌNH =====
window_width = 700
window_height = 500
screen_width = root.winfo_screenwidth() # Lấy độ rộng màn hình
screen_height = root.winfo_screenheight() # Lấy chiều cao màn hình
x = (screen_width // 2) - (window_width // 2) # Tính vị trí x để căn giữa
y = (screen_height // 2) - (window_height // 2) # Tính vị trí y để căn giữa
root.geometry(f"{window_width}x{window_height}+{x}+{y}")
root.config(bg="#f7f7f7")
root.resizable(False, False)
```

```
# ===== MENU =====
menu = tk.Menu(root) # Tạo menu chính
root.config(menu=menu)
frames = {} # Lưu tất cả các Frame tương ứng với từng trang

# Danh sách các bảng cần quản lý
tables = ["diem", "sinhvien", "lop", "khoa", "giangvien", "monhoc"]
# Tên tiêu đề tương ứng từng bảng
titles = {
    "diem": "QUẢN LÝ ĐIỂM SINH VIÊN",
    "sinhvien": "DANH SÁCH SINH VIÊN",
    "lop": "DANH SÁCH LỚP",
    "khoa": "DANH SÁCH KHOA",
    "giangvien": "DANH SÁCH GIẢNG VIÊN",
    "monhoc": "DANH SÁCH MÔN HỌC"
}
```

```

# ===== FONT CHUNG =====
title_font = ("Arial", 18, "bold") # Font
# ===== TẠO FRAME CHO TỪNG BẢNG =====
for t in tables:
    frames[t] = tk.Frame(root, bg="#fff")
    title_label = tk.Label( #Tiêu đề
        frames[t],
        text=titles[t],
        font=title_font,
        fg="#333",
        bg="#fff"
    )
    title_label.pack(pady=(20, 5)) # Thêm

```

```

if t == "diem": #Quản lý trang điểm
    form = tk.Frame(frames[t], bg="#fff") #Form nhập dữ liệu
    form.pack(pady=20, padx=100, fill="x") #fill="x" cho form giúp giãn ngang.
    # Text mã số sinh viên
    tk.Label(form, text="Mã số sinh viên:", bg="#fff").grid(row=0, column=0, padx=10, pady=5, sticky="w")
    maSV_entry = tk.Entry(form, width=20)
    maSV_entry.grid(row=0, column=1, padx=5, pady=5, sticky="w")
    # Combobox mã môn học
    tk.Label(form, text="Mã môn học:", bg="#fff").grid(row=0, column=2, padx=10, pady=5, sticky="w")
    # --- Kết nối CSDL và lấy danh sách mã môn học ---
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("SELECT maMH FROM monhoc")
    maMH_list = [row[0] for row in cur.fetchall()]
    conn.close()
    # --- Tạo Combobox hiển thị mã môn học ---
    maMH_entry = ttk.Combobox(form, values=maMH_list, width=18, state="readonly")
    maMH_entry.grid(row=0, column=3, padx=5, pady=5, sticky="w")
    # Text điểm quá trình
    tk.Label(form, text="Điểm quá trình:", bg="#fff").grid(row=1, column=0, padx=10, pady=5, sticky="w")
    diemQT_entry = tk.Entry(form, width=20)
    diemQT_entry.grid(row=1, column=1, padx=5, pady=5)
    # Text điểm thi
    tk.Label(form, text="Điểm thi:", bg="#fff").grid(row=1, column=2, padx=10, pady=5, sticky="w")
    diemThi_entry = tk.Entry(form, width=20)
    diemThi_entry.grid(row=1, column=3, padx=5, pady=5)

```

```

# --- HÀM TÌM KIẾM THEO MÃ SINH VIÊN ---
def tim_kiem_diem():
    try:
        maSV = maSV_entry.get().strip()
        if not maSV:
            messagebox.showwarning("Cảnh báo", "Vui lòng nhập mã sinh viên cần tìm!")
            return

        conn = connect_db()
        cur = conn.cursor()
        cur.execute("""
            SELECT d.maSV, mh.maMH, d.diemQT, d.diemThi, d.diemTong
            FROM diem d
            JOIN monhoc mh ON d.maMH = mh.maMH
            WHERE d.maSV = %s
        """, (maSV,))
        rows = cur.fetchall()
        conn.close()

        # Xóa dữ liệu cũ trong TreeView
        for item in tree_diem.get_children():
            tree_diem.delete(item)

        # Cập nhật lại cột
        columns = ["Mã SV", "Tên Môn Học", "Điểm QT", "Điểm Thi", "Điểm Tổng"]
        tree_diem["columns"] = columns
        for col in columns:
            tree_diem.heading(col, text=col)
            tree_diem.column(col, width=150, anchor="center")

```

```

        # Thêm dữ liệu tìm thấy
        for row in rows:
            tree_diem.insert("", "end", values=row)

        if not rows:
            messagebox.showinfo("Kết quả", f"Không tìm thấy điểm cho sinh viên có mã: {maSV}")

    except Exception as e:
        messagebox.showerror("Lỗi", str(e))

```

```

# ===== HÀM RESET =====
def reset_fields():
    maSV_entry.delete(0, tk.END)
    maMH_entry.set("") # Xóa chọn combobox (đưa về trống)
    diemQT_entry.delete(0, tk.END)
    diemThi_entry.delete(0, tk.END)

```

```
# --- NÚT CHỨC NĂNG ---
btn_frame = tk.Frame(frames[t], bg="#ffff")
btn_frame.pack(pady=2)
ttk.Button(btn_frame, text="Thêm điểm", width=15, command=lambda: them_diem()).grid(row=0, column=0, padx=8, pady=8)
ttk.Button(btn_frame, text="Sửa điểm", width=15, command=lambda: sua_diem()).grid(row=0, column=1, padx=8, pady=8)
ttk.Button(btn_frame, text="Xóa điểm", width=15, command=lambda: xoa_diem()).grid(row=0, column=2, padx=8, pady=8)
ttk.Button(btn_frame, text="Tính điểm TB & xếp loại", width=25, command=lambda tr=None: calc_average(tree_diem)).grid(row=0, column=3, padx=8, pady=8)
ttk.Button(btn_frame, text="Tải danh sách", width=15, command=lambda n=t: load_data("diem", tree_diem)).grid(row=0, column=4, padx=8, pady=8)
btn_frame2 = tk.Frame(frames[t], bg="#ffff")
btn_frame2.pack(pady=2)
ttk.Button(btn_frame2, text="Lưu điểm", width=18, command=lambda: luu_excel()).pack(side="left", padx=10, pady=5)
ttk.Button(btn_frame2, text="Reset", width=18, command=reset_fields).pack(side="left", padx=10, pady=5)
ttk.Button(btn_frame2, text="Tìm kiếm", width=18, command=tim_kiem_diem).pack(side="left", padx=10, pady=5)
ttk.Button(btn_frame2, text="Thoát", width=18, style="Accent.TButton", command=root.destroy).pack(side="left", padx=10, pady=5) #Sử dụng grid cho hàng 1
```

```
# --- BẢNG DỮ LIỆU ---
tree_diem = ttk.Treeview(frames[t], show="headings") #show=
tree_diem.pack(fill="both", expand=True, padx=10, pady=10)
def select_item(event): #Khi nhả chuột trên một hàng, lấy i
    selected = tree_diem.focus()
    if not selected:
        return
    values = tree_diem.item(selected, "values")
    if len(values) >= 4:
        maSV_entry.delete(0, tk.END)
        maSV_entry.insert(0, values[0])
        maMH_entry.set(values[1])
        diemQT_entry.delete(0, tk.END)
        diemQT_entry.insert(0, values[2])
        diemThi_entry.delete(0, tk.END)
        diemThi_entry.insert(0, values[3])
    tree_diem.bind("<ButtonRelease-1>", select_item)
#Điền giá trị vào các Entry để sửa hoặc xóa thuận tiện.
```

```
# ===== HÀM THÊM =====
def them_diem():
    try:
        maSV = maSV_entry.get()
        maMH = maMH_entry.get()
        diemQT = float(diemQT_entry.get()) #Lấy dữ liệu từ Entry, ép kiểu float cho điểm.
        diemThi = float(diemThi_entry.get())
        diemTong = diemQT * 0.4 + diemThi * 0.6

        conn = connect_db()
        cur = conn.cursor()
        cur.execute("INSERT INTO diem (maSV, maMH, diemQT, diemThi) VALUES (%s, %s, %s, %s)", (maSV, maMH, diemQT, diemThi))
        conn.commit()
        conn.close()
        messagebox.showinfo("Thành công", "Đã thêm điểm!")
        load_data("diem", tree_diem) #Gọi load_data("diem", tree_diem) để refresh bảng hiển thị sau khi thao tác.
    except Exception as e:
        messagebox.showerror("Lỗi", str(e))
```

```
# ===== HÀM SỬA =====
def sua_diem(): #dùng UPDATE ... WHERE maSV=%s AND maMH=%s để cập nhật điểm.
    try:
        maSV = maSV_entry.get()
        maMH = maMH_entry.get()
        diemQT = float(diemQT_entry.get())
        diemThi = float(diemThi_entry.get())
        diemTong = diemQT * 0.4 + diemThi * 0.6
        conn = connect_db()
        cur = conn.cursor()
        cur.execute("UPDATE diem SET diemQT=%s, diemThi=%s WHERE maSV=%s AND maMH=%s", (diemQT, diemThi, maSV, maMH))
        conn.commit()
        conn.close()
        messagebox.showinfo("Thành công", "Đã cập nhật điểm!")
        load_data("diem", tree_diem) #gọi load_data sau khi commit để cập nhật UI.
    except Exception as e:
        messagebox.showerror("Lỗi", str(e))
```

```
# ===== HÀM XÓA =====
def xoa_diem(): #dùng DELETE FROM diem WHERE maSV=%s AND maMH=%s để xóa.
    try:
        maSV = maSV_entry.get()
        maMH = maMH_entry.get()
        conn = connect_db()
        cur = conn.cursor()
        cur.execute("DELETE FROM diem WHERE maSV=%s AND maMH=%s", (maSV, maMH))
        conn.commit()
        conn.close()
        messagebox.showinfo("Thành công", "Đã xóa điểm!")
        load_data("diem", tree_diem) #gọi load_data sau khi commit để cập nhật UI.
    except Exception as e:
        messagebox.showerror("Lỗi", str(e))
```

```

# ==== HÀM LƯU RA FILE EXCEL ====
def luu_excel():
    try:
        # Hộp thoại chọn nơi lưu
        # asksaveasfilename() -> mở hộp thoại "Save As" cho người dùng
        # defaultextension=".xlsx": nếu không gõ đuôi, tự thêm .xlsx
        # filetypes: lọc chỉ hiển thị file Excel
        # title: tiêu đề cửa sổ
        file_path = filedialog.asksaveasfilename(
            defaultextension=".xlsx",
            filetypes=[("Excel files", "*.xlsx")],
            title="Chọn nơi lưu file Excel"
        )
        if not file_path: #kết quả trả về file_path nếu người dùng bấm cancel thì return
            return # nếu người dùng bấm Cancel

        # Kết nối DB & lấy dữ liệu
        conn = connect_db()
        cur = conn.cursor()
        cur.execute("""
            SELECT d.maSV, sv.hoTen, d.maMH, mh.tenMH, d.diemQT, d.diemThi, d.diemTong
            FROM diem d
            JOIN sinhvien sv ON d.maSV = sv.maSV
            JOIN monhoc mh ON d.maMH = mh.maMH
            """)
        rows = cur.fetchall()
        conn.close()

        # Ghi vào file Excel
        workbook = xlswriter.Workbook(file_path) #Tạo một workbook
        worksheet = workbook.add_worksheet("Điểm sinh viên") #thêm một sheet
        bold = workbook.add_format({'bold': True, 'bg_color': '#D9E1F2'}) #Tạo định dạng

```

```

# Tiêu đề cột
headers = ["Mã SV", "Họ tên", "Mã môn học", "Tên môn học", "Điểm QT", "Điểm Thi", "Điểm Tổng"]
for col, header in enumerate(headers): #Lặp qua từng header cùng chỉ số cột col (bắt đầu từ 0)
    worksheet.write(0, col, header, bold) #Ghi tiêu đề vào hàng 0 (dòng đầu tiên trong Excel)
    worksheet.set_column(col, col, 18) #Đặt độ rộng của cột

# Ghi dữ liệu
for row_num, row_data in enumerate(rows, start=1):
    for col_num, value in enumerate(row_data):
        worksheet.write(row_num, col_num, value)

workbook.close() #Đóng workbook và ghi thực sự file .xlsx ra đĩa
messagebox.showinfo("Thành công", f"Đã lưu dữ liệu ra file:\n{file_path}")

except Exception as e:
    messagebox.showerror("Lỗi", str(e))

```

```

else:

    # === Các trang khác ===
    tree = ttk.Treeview(frames[t], show="headings")
    tree.pack(fill="both", expand=True, padx=10, pady=10)
    ttk.Button(frames[t], text="Tải danh sách", command=lambda n=t, tr=tree: load_data(n, tr)).pack(pady=5)

```

```
# ===== HÀM CHUYỂN FRAME =====
def show_frame(name):
    for f in frames.values():
        f.pack_forget() # ẩn frame khỏi cửa sổ
    frames[name].pack(fill="both", expand=True)
```

```
# ===== HÀM TẢI DỮ LIỆU =====
def load_data(table_name, tree):
    for item in tree.get_children():
        tree.delete(item)
    conn = connect_db() #mở kết nối
    cursor = conn.cursor() #tạo curs
```

```
query_map = { #là dict ánh xạ tên logical (vd "sinhvien") sang câu SQL tương ứng. Giúp hàm chung xử lý nhiều bảng.
    "khoa": "SELECT maKhoa AS 'Mã Khoa', tenKhoa AS 'Tên Khoa' FROM khoa",
    "lop": "SELECT maLop AS 'Mã Lớp', tenLop AS 'Tên Lớp', maKhoa AS 'Mã Khoa' FROM lop",
    "giangvien": "SELECT maGV AS 'Mã Giảng Viên', hoTen AS 'Họ Tên Giảng Viên', maKhoa AS 'Mã Khoa' FROM giangvien",
    "sinhvien": "SELECT maSV AS 'Mã Số Sinh Viên', hoTen AS 'Họ Tên', ngaySinh AS 'Ngày Sinh', gioiTinh AS 'Giới Tính', diaChi AS 'Địa Chỉ', maLop AS 'Mã Lớp' FROM sinhvien",
    "monhoc": "SELECT maMH AS 'Mã Môn Học', tenMH AS 'Tên Môn Học', soTinChi AS 'Số Tín Chỉ', maGV AS 'Giảng Viên Phụ Trách' FROM monhoc",
    "diem": "SELECT maSV AS 'Mã Số Sinh Viên', maMH AS 'Mã Môn Học', diemQT AS 'Điểm Quá Trình', diemThi AS 'Điểm Thi', diemTong AS 'Điểm Tổng' FROM diem"
}

cursor.execute(query_map[table_name])
rows = cursor.fetchall()
#Thực hiện truy vấn và lấy toàn bộ dữ liệu về.

columns = [desc[0] for desc in cursor.description] #
tree["columns"] = columns #đặt cấu hình cột cho Tree
for col in columns:
    tree.heading(col, text=col) #đặt tiêu đề hiển th
    tree.column(col, width=150, anchor="center") #đặ

for row in rows:
    tree.insert("", "end", values=row) #thêm từng hà
conn.close() #đóng kết nối (rất quan trọng để giải p
```

```
# ===== HÀM TÍNH ĐIỂM TB & XẾP LOẠI =====
def calc_average(tree):
    for item in tree.get_children():
        tree.delete(item)
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT sv.maSV, sv.hoTen, AVG(d.diemTong) AS diemTB,
        CASE
            WHEN AVG(d.diemTong) >= 8 THEN 'Giỏi'
            WHEN AVG(d.diemTong) >= 6.5 THEN 'Khá'
            WHEN AVG(d.diemTong) >= 5 THEN 'Trung bình'
            ELSE 'Yếu'
        END AS xepLoai
        FROM diem d
        JOIN sinhvien sv ON d.maSV = sv.maSV
        GROUP BY sv.maSV, sv.hoTen
    """)
    rows = cursor.fetchall()
    columns = ["Mã SV", "Họ Tên", "Điểm TB", "Xếp loại"]
    tree["columns"] = columns
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=150, anchor="center")
    for row in rows:
        diem_tb = round(row[2], 2) if row[2] is not None else 0 # làm tròn
        tree.insert("", "end", values=(row[0], row[1], diem_tb, row[3]))
    conn.close()
```

```
# ===== MENU CHUYỂN TRANG =====
submenu = tk.Menu(menu, tearoff=0) #Tạo một submenu trong thanh menu chính.
menu.add_cascade(label=" Trang chính", menu=submenu)
for t in tables:
    submenu.add_command(label=titles[t], command=lambda n=t: show_frame(n))
show_frame("diem") #hiển thị trang mặc định.

root.update() #cập nhật giao diện trước khi lấy kích thước.
root.minsize(root.winfo_width(), root.winfo_height()) #đặt kích thước tối thiểu
root.mainloop() #bắt đầu vòng lặp sự kiện Tkinter – GUI phản hồi người dùng
```

PHỤ LỤC

Phụ lục A: Mã SQL tạo cơ sở dữ liệu (QuanLy_DiemSinhVien.sql)

Phụ lục B: Sơ đồ ERD (Cơ sở dữ liệu.png)

Phụ lục C: Mã nguồn Python đầy đủ (Database_ERM_cosodulieu.mwb)

TÀI LIỆU THAM KHẢO

Nguyễn Ngọc Minh (2025). Hướng dẫn thực hành và yêu cầu đồ án Python (COS525). Trường Đại học An Giang.

Nguyễn Thị Mỹ Linh (2020). Giáo trình Cơ sở dữ liệu. Nhà xuất bản Giáo dục Việt Nam.

Đại học CNTT TP.HCM (2022). Phân tích thiết kế hệ thống thông tin. NXB ĐHQG TP.HCM.

Oracle Corporation (2024). MySQL 8.0 Reference Manual. Oracle Docs.

Python Software Foundation (2023). Tkinter GUI Programming Documentation.